Security measures that stop the other types of intruders affect the techniques used by skilled attackers. If you've ditched that unsecured inbound access method, the intruder can't find it. If your servers and programs are up to date and correctly configured, the intruder will need to find a previously unknown exploit to break into your network. If a skilled intruder really wants *your* company's data, he will need to change tactics. Maybe he will try dumpster diving for old sticky notes, or even show up dressed as a telco repairman and try to install a packet sniffer. If an intruder knows everything about your network and his easiest way to break in is *still* something out of a caper film, your security is pretty good.

NOTE *The word hacker has different meanings depending on who is talking. In the technical world, a hacker is someone not only interested in the inner workings of technology but also capable of creating new technology. The media has transformed the word to mean "someone who breaks into computers." I recommend completely avoiding the word "hacker," and using terms like "intruder" or "gravy sucking pig-dog" instead. When to use each is up to you, of course.*

OpenBSD Security Announcements

Your best line of defense against all types of intruders is keeping your computer software up to date. This means you need to know when to update your system and what to update. The OpenBSD Project maintains a low-traffic mailing list, *security-announce@OpenBSD.org*, specifically to broadcast new security alerts to users. Subscribe to this list.

If you don't feel like subscribing to yet another mailing list, these security alerts are also posted on OpenBSD-specific sites such as *http://www.undeadly.org*.

Note that this won't get you security alerts for third-party software running on OpenBSD. You must get updates for those programs separately. Check the software's website for details on how to get their security announcements. All the time you've spent securing your operating system will be wasted if someone hijacks the insecure web application you neglected to update.

OpenBSD Memory Protection

One of the most common intrusion paths is to attack what's in the computer's memory. If intruders can access memory that they shouldn't be able to access, or if they can make a program access memory it shouldn't, they have any number of ways to get into the system.

OpenBSD includes a whole bunch of security features for system memory that the sysadmin never actually sees. You don't need to turn on the nonexecutable stack; it's just there.

Some of these features appear only in OpenBSD. Some appeared first in OpenBSD, and then spread elsewhere. Some came from research papers. Others build on hardware features.

The OpenBSD team takes a more proactive attitude about security features than many other projects. As an example, consider the ProPolice deployment several years ago. ProPolice is a compiler feature that prevents certain classes of buffer overflows. When you enabled ProPolice in the early days, a lot of software could not be built. Even more software could be built, but it crashed when used. These failures were not ProPolice problems. ProPolice simply exposed programming errors in the software. But many users and developers said that "enabling ProPolice breaks all kinds of stuff, so don't turn it on."

The OpenBSD team enabled ProPolice by default in a development snapshot. What happened? Stuff—a lot of stuff—broke. Many third-party applications needed by OpenBSD users either could not build or would not run. Third-party application vendors started receiving bug reports from OpenBSD users who were able to say exactly how the software was broken. Software vendors started fixing bugs.

ProPolice didn't cause these crashes; it merely exposed bugs. By enabling ProPolice by default, OpenBSD gave the free software world incentive to fix those bugs. Eventually, as the type of bugs revealed by ProPolice became less common, other operating systems also enabled ProPolice. OpenBSD's willingness to take this step improved computer security as a whole.

If you closely follow OpenBSD development, expect to see more of this behavior. The OpenBSD team does what it considers most correct, not what is most convenient or easiest.

The common memory security features you should know about include W^X, .rodata segments, guard pages, randomized memory allocations, ProPolice, and protecting `atexit` and `stdio`. We'll cover each in turn.

W^X

W^X stands for Write Xor Execute. Once a program is loaded, that program's pages in memory are either writable or executable, but not both.

A common exploit technique is to trick a program into writing information to memory, and then executing that piece of memory. An attacker might convince a program to write to a chunk of memory, but the kernel will not allow that memory to be executed.

Some hardware platforms (such as amd64) have hardware support for W^X. If that support exists, OpenBSD uses it.

.rodata Segments

A segment of memory containing program code traditionally had two parts: actual code and read-only data, or *.rodata segments*. In the past, some operating systems allowed programs to modify read-only memory. OpenBSD prevents this by leveraging hardware features when available.

Guard Pages

Many pieces of software used to access memory beyond what they allocated. If a program writes to memory that doesn't belong to it, it's writing to memory that belongs to a different program. Intruders use this to exploit programs. A *guard page* is a single page of memory next to the memory allocated by a program. The program cannot write to this memory. If the program tries to write to the guard page, it probably will crash. By enforcing this limit, OpenBSD protects other programs.

Using guard pages everywhere would use a lot of memory, so OpenBSD enables guard pages only in carefully selected places.

Address Space Layout Randomization

Traditionally, computers allocate memory consecutively. This can give intruders certain advantages. If they know that program A usually loads after program B, and they know they can make program B write to memory space outside its allocation, they can guess that they can write to program A's memory space and make program A fail in a predictable manner. Doing so requires a certain degree of skill, but once one person figures out this exploit, innumerable people can use it.

OpenBSD randomizes where it allocates memory. Two programs started one after the other don't get consecutive memory blocks. The randomization is done intelligently, to avoid wasting memory. Intruders cannot use one program against another in this manner.

ProPolice

ProPolice protects code against attacks that manipulate the memory stack. When code is compiled, ProPolice adds additional code to keep a program within its own area of memory. If ProPolice determines that specific areas of memory (called *canaries*) have been changed, it immediately aborts the program. Where other memory protection techniques prevent writing to executable memory, ProPolice terminates a process when writable memory that *can* be written to, but specifically *should not*, is changed.

And More!

OpenBSD includes a whole bunch of small memory guards scattered throughout. Here's a small sampling:

- The `malloc()` and `atexit()` system calls mark memory nonwritable after updating it.
- File descriptor handling has been carefully audited throughout.
- `snprintf` is async-signal-safe when no floats are involved.

And the list continues.

Could any of these be exploited in the real world? Some of them have, and some are just theoretical. But I would rather be protected against theoretical threats than assume no one can break something that has never been broken before.

File Flags

All Unix-like operating systems share a common permissions scheme, but OpenBSD (and most BSD-based operating systems) extends the permissions scheme with *file flags*. File flags work with permissions to change file security. Flags can make a file unchangeable, make it so that existing data cannot be removed and users can only add to the file, and produce several other effects. Some flags have functions unrelated to security, but we'll pay special attention to the security flags. File flags are listed and documented in `chflags(1)`.

File Flag Types

Many file flags have different effects depending on the system securelevel, which we'll cover in the next section. Understanding how securelevels work requires an understanding of file flags, while file flags rely on securelevels. For the moment, just nod and smile when I mention securelevels while discussing file flags. All will become clear, trust me.

OpenBSD's UFS and UFS2 filesystems support the following file flags:

sappnd
: Files with the system-level append-only flag can be added to but cannot be removed or otherwise edited. The `sappnd` flag is particularly useful for log files. For example, a common intruder tactic is to remove *.history* or symlink it to */dev/null* so that the administrator cannot see what happened. Setting `sappnd` on a user's *.history* file can be interesting if the account is compromised. Using the `sappnd` flag ensures that intruders cannot cover their tracks in this manner. Only root can set or remove the `sappnd` flag, and it cannot be removed when the system is running at securelevel 1 or higher.

uappnd
: The user-level append-only flag can be set only by the file owner or root. As with `sappnd`, a file with the `uappnd` flag can be added to but not otherwise edited or removed. This is most useful for personal logs and files; it primarily adds an extra layer of protection against users accidentally deleting their own files. The owner or root can set or remove this flag.

schg
: Files with the system-level immutable flag cannot be changed in any way. They cannot be edited, moved, replaced, or overwritten. Basically,

the filesystem itself prevents all attempts to alter this file. Only root can set or remove this flag, and it cannot be removed when the system is running at securelevel 1 or higher.

uchg
: The user-level immutable flag prevents anyone from changing the file. It's a user-level flag, so root can override it. This flag helps to prevent a file from being edited or removed by accident, but it's not a way to secure the system. The owner or root can set or remove this flag at any securelevel.

nodump
: The no dump flag tells `dump(8)` to not back up a file. Set this on files that don't need to be backed up to tape. Check your backup program's documentation to see if it honors this flag.

Setting, Viewing, and Removing File Flags

Set file flags with `chflags(1)`. For example, if you are really worried about someone changing your kernel file, you could mark */bsd* with the system-level immutable flag.

```
# chflags schg /bsd
```

This would prevent anyone—including you—from changing the kernel, reconfiguring the kernel, or upgrading the system.

You can also recursively change the file flags on an entire directory tree with the `-R` flag. If you wanted to make the entirety of */bin* immutable, you would run this command:

```
# chflags -R schg /bin
```

And poof, you can no longer upgrade your system.

To view the flags on a file, use `ls -lo`.

```
$ ls -lo vitallog
-rw-r--r-- 1 root wheel - 20915343 Jul 17 16:56 vitallog
```

This file has no flags set on it. Let's set the system-level append-only flag.

```
$ chflags sappnd vitallog
chflags: vitallog: Operation not permitted
```

Oh, right—only root can set system-level flags. Let's try again:

```
$ sudo chflags sappnd vitallog
Password:
$ ls -lo vitallog
-rw-r--r-- 1 mwlucas mwlucas sappnd 20915343 Jul 17 16:56 vitallog
```

This file now has the sappnd flag. The system can add to it, but cannot otherwise edit or remove it.

OpenBSD doesn't flag any files out of the box, so you'll need to add flags yourself if you want them. Before you go nuts, however, note that adding file flags increases the overhead for system maintenance. If upgrading a system is hard, the sysadmin won't want to do it. Is it more secure to have all your programs in */bin* immutable, or is it more secure to simplify upgrades, updates, and application of security patches?

To remove a flag from a file, use chflags with a no before the flag name. For example, to unset the sappnd flag on the *vitallog* file, try this:

```
$ sudo chflags noschg vitallog
Password:
chflags: vitallog: Operation not permitted
```

Wait a minute! I'm running under sudo(8), and I have root-level privileges. What's going on?

By default, OpenBSD runs at securelevel 1. When running at securelevel 1 or higher, you cannot unset system-level file flags, so an attempt to do so failed. You can remove these flags only at securelevel -1 or in single-user mode. Read on to learn about securelevels.

Securelevels

securelevel(7) is a kernel setting to restrict actions the system can perform. The kernel behaves slightly differently as you raise the securelevel. For example, at low securelevels, the file flags discussed in the previous section can be removed; a file might be marked immutable, but you can remove the marker, delete or edit the file, and restore the flag. When you increase the securelevel, however, you can no longer remove the flag. Similar changes take place in other parts of the system. Taken as a whole, these changes might frustrate or stop an intruder.

Securelevel settings range from -1 to 2. Though OpenBSD runs at securelevel 1 by default, you can change this setting to fit your environment.

Higher securelevels make system maintenance difficult. Many actions taken during normal upgrades and administration are also things that intruders might do to cover their tracks. It might make sense for you to run a highly secure, stable server at securelevel 2, and run your experimental machine at -1. On the other hand, the OpenBSD folks don't encourage changing from the default securelevel. Running your system at -1 may leave you open to attacks, while running at 2 complicates management and maintenance. Which securelevel you choose depends on your environment.

Despite the name, a securelevel is not an all-purpose general security dial. Arbitrarily increasing the securelevel will do nothing but annoy you and your users. While you can increase the securelevel at any time, you cannot reduce the securelevel without rebooting the system, so don't experiment blindly.

Setting the System Securelevel

Set the boot-time securelevel in */etc/rc.securelevel*. In that file, you'll find a line like this:

```
securelevel=1
```

Change the `1` to your preferred securelevel. On your next reboot, the system will go to this securelevel when it enters multiuser mode. If you need to run a process before the boot process raises the securelevel, put the command to start the process in this file.

If you want to raise the securelevel without rebooting, adjust the `kern.securelevel sysctl(3)` to the desired value.

```
# sysctl kern.securelevel=2
kern.securelevel: 1 -> 2
```

Remember that you cannot lower the securelevel of a running system. If a sysadmin could lower the securelevel, so could an intruder.

Securelevel Definitions

OpenBSD has four securelevels: -1, 0, 1 and 2. We'll cover each in turn.

Securelevel -1

Securelevel -1 is also called permanently insecure mode. The system isn't necessarily insecure—it's just that none of the securelevel protections are in place. I use securelevel -1 only to remove file flags that I never should have used in the first place.

Securelevel 0

Securelevel 0 is used only when the system is first booting. It offers no special features. When the system reaches multiuser mode, however, the securelevel is automatically raised to 1. Setting `securelevel=0` in */etc/rc.securelevel* is functionally equivalent to setting `securelevel=1`.

Securelevel 1

At securelevel 1, OpenBSD's default, things become interesting.

The securelevel affects certain kernel configuration settings, called sysctls (covered in Chapter 18). Early in the boot process, OpenBSD uses the settings in */etc/sysctl.conf* to set sysctls. When I say that a particular sysctl cannot be changed, read that as "without altering the configuration and rebooting."

Securelevel 1 implements the following limitations:

- No one can write to the */dev/mem* and */dev/kmem* devices. Many security exploits work by writing to these devices.

- The raw disk devices of all mounted file systems are read-only. (Writing to the raw devices of mounted filesystems would let you change files without regard to permissions.) Programs should access mounted filesystems only through the filesystem anyway, so this won't change day-to-day operations.
- The system-level file flags `schg` and `sappnd` cannot be removed.
- Kernel modules cannot be loaded or unloaded. OpenBSD supports kernel modules, but the default kernel is monolithic. There's no legitimate reason to load a kernel module on a running production system.
- The sysctl `fs.posix.setuid` cannot be changed. By default, `chown(1)` clears the `setuid` and `setgid` bits on files when changing file permissions, as per the POSIX standard. You can override this by setting `fs.posix.setuid` to 0.
- The sysctl `hw.allowpowerdown` cannot be changed. This controls the power button's interaction with the system. When it's set to 1, briefly pressing the power button shuts down the system cleanly. When it's set to 0, the power button does not shut down the system. (You can still shut down the system by holding down the power button for several seconds, but that's not a clean shutdown.) Not all platforms support this kind of shutdown or power management.
- The sysctl `net.inet.ip.sourceroute` cannot be changed. Source routing is a technique to permit the sender of a packet to control which route the packet takes across the network. It's caused many security problems, and its use is generally discouraged. OpenBSD ignores source routing by default. Setting `net.inet.ip.sourceroute` to 1 forces OpenBSD to pay attention to source routing.
- The sysctl `machdep.kbdreset` cannot be changed. When set to 1, `machdep.kbdreset` allows the system to be cleanly rebooted using CTRL-ALT-DELETE. When this sysctl is set to 0, the system ignores CTRL-ALT-DELETE.
- The `ddb.console` and `ddb.panic` sysctls may not be raised. Raising these sysctls enables certain kernel debugging options. Unauthorized users with physical access could gain unlimited system access through the debugger if they could raise these sysctls.
- The sysctl `machdep.allowaperture` cannot be raised. If you want to use the X Window System (discussed in Chapter 17), you must allow X access to specific parts of kernel memory by enabling this sysctl early during the boot process. If you're not running X, no one legitimately needs this access.
- General-purpose input/output (GPIO) controllers cannot be further configured. GPIO controllers support a wide variety of special-purpose hardware. See `gpio(4)` and `gpioctl(8)` for details on each.

These limitations have little effect on normal day-to-day operations, but they can interfere with debugging. If you're trying to discover why your GPIO device isn't working, you probably want to set your securelevel to -1.

Securelevel 2

Securelevel 2 is the highest securelevel in OpenBSD, and it disables a variety of features that you might need during normal maintenance. Use securelevel 2 only on a stable machine that you don't expect to change much. If you need to change anything restricted by securelevel 2, you must reboot the machine.

Securelevel 2 includes everything from securelevel 1, plus the following:

- Raw disk devices are always read-only. You cannot format, fdisk, or disklabel disks, even if they're not in use.
- The system clock cannot be moved backward, nor close to the overflow point. Make sure your system time is correct before entering multiuser mode!
- You cannot alter packet-filtering rules (covered in Chapters 21 and 22). Packet filters, network address translation (NAT), traffic queues, and so on are immutable.
- Kernel debugger sysctl values (those beginning with `ddb`) cannot be changed.

So, for example, you don't want your firewall at securelevel 2 unless you understand that packet filtering rules can change only with a reboot.

What Securelevel Do You Need?

The securelevel appropriate for your environment depends entirely on your situation, but the overwhelming majority of the time, the default of securelevel 1 is most suitable.

If you are debugging and testing features and tools, you might find that you need to use securelevel -1 on a development machine. Once you've worked out how to configure your GPIO device or the correct debugger settings for your system, however, use securelevel 1 so that you mirror a production environment.

If you have a very stable system, you could try securelevel 2. In all my years of running OpenBSD, though, I've had only one system for which securelevel 2 was the right choice, and several cases where securelevel 2 created more trouble than it was worth.

Securelevel Weaknesses

What can't securelevels do? Consider a case where someone compromises a web application on your server, uses that to bootstrap himself into a shell, and then uses the shell to bootstrap himself into root access. Securelevels don't do anything to prevent this.

Unless you've made copious use of the `schg` flag, the intruder can replace system binaries with ones that send your authentication credentials to a free email account registered in a bogus name. So you decide to run around applying the `schg` flag to the contents of critical directories like */bin* and */usr/lib*. That will stop the bugger! Well, that will work as long as you make

every file immutable, including the system configuration files in */etc*—you know, the ones that you need to change in order to do your job. If you leave one file unprotected, the intruder could add a command like `chflags -R noschg /` to an early part of the system startup, and poof—the next time you reboot your system, you unlock all your files. How often do you exhaustively audit your */etc* files? And you'll need to undo this tangled morass every time you patch or upgrade your system!

This is only one possible path. There are many ways for an intruder to lever himself into the system. Relying on securelevels to protect you is unwise. Use them and consider them a tool in your kit, but don't think they are a panacea for every problem.

Keeping Secure

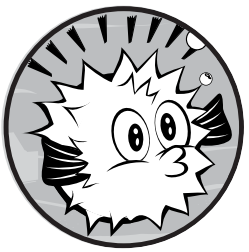
The tools discussed in this chapter are not OpenBSD's only security features. The OpenBSD team has put a lot of work into securing every part of the system. But this chapter covers some things that make OpenBSD special and gives you an idea of how those features work.

What's the best path to security? Keep your system updated and configure your server daemons securely. It's boring, but it works.

11

OVERVIEW OF TCP/IP

IP version 6:
All the pain of version 4,
plus brand-new problems!



Securing a computer is easy: Disconnect it from all networks, remove all input and output, and lock it in a bunker. Oh, wait—did you want the system to *do* something? Then you'll probably want to connect your system to the Internet.

Many system administrators have a vague familiarity with the basics of networking, but to be a truly competent sysadmin, you need a real understanding of how everything fits together. You don't need to know when to use rapid spanning trees, how to choose between BGP and OSPF, or even what those acronyms represent. But you must know what an IP address is, how a netmask works, how port numbers differ from protocol numbers, and why you cannot use `telnet(1)` to test UDP connectivity. Without this basic knowledge, you'll fumble. Read this chapter and understand it, and you'll have an easier time convincing your network administrator to give you what you need.

While this chapter offers an overview of TCP/IP, it doesn't cover the innumerable details, caveats, annoyances, peccadilloes, and blatant outrages present in the protocol. If you find that you need to torture yourself with the finer points of TCP/IP, pick up one of the big, thick books on the subject. *The TCP/IP Guide* by Charles M. Kozierok (No Starch Press, 2005) is an excellent place to start.

This chapter covers both TCP/IP version 4 (the Internet protocol widely used for the last 30-odd years) and the new version of the protocol, TCP/IP version 6. Despite the different version numbers, the two protocols are more similar than not.

We'll start with the layers of the network and then delve into how the protocols work.

Network Layers

The network protocol is divided into several layers. Each layer handles a specific task and interacts only with the layers immediately above and below it. At first, you might laugh at the idea that this layer model simplifies the network process, but it really does. The important thing to remember right now is that each layer communicates with only the layer directly above it and the layer directly beneath it (theoretically, anyway).

The classic Open Systems Interconnection (OSI) network protocol stack represents the network as seven layers. It's an exhaustively complete model and covers almost any situation using any network protocol and any application. Because the Internet is a very specific type of network, and because this isn't a book about networking or networked applications in general, I'll limit my discussion of TCP/IP to four specific layers of the network: physical, datalink, network, and transport. Don't worry—these four layers cover the Internet and (almost) all corporate networks.

The Physical Layer

Whether it's copper or fiber-optic cable, or even radio waves, physical wire is a layer of the network. Without some physical media to run over, a network cannot function. Everything from the CAT5 cable plugged into your desktop to the fiber-optic cable connecting you to Asia is part of the physical layer. If it can be tripped over, backhoed, or interfered with, it's part of the physical layer. For simplicity's sake, I'll refer to the physical layer as the *wire*, although it can take innumerable forms.

This is the easiest layer to understand. If your wire meets the requirements of the physical protocol, you're in business. If not, your network won't work. One of the functions of Internet routers is to connect one sort of physical layer to another—for example, converting local Ethernet into an OC3 fiber connection.

The physical layer has no decision-making abilities of its own; everything that runs over it is dictated by the datalink layer.

The Datalink Layer

The datalink layer is the protocol that runs over the physical wire. It transforms information into the actual signals that are sent over the physical layer, using the appropriate encoding for that physical media, as follows:

- Both Ethernet and Switched Multimegabit Data Service (SMDS) use Media Access Control (MAC) addresses and the Address Resolution Protocol (ARP).
- IPv6 over Ethernet uses Neighbor Discovery (ND).
- Dial-up and wide area networks (WANs) use either the Point-to-Point Protocol (PPP) or High-Level Data Link Control (HDLC).

OpenBSD supports other common datalink protocols, such as PPP over Ethernet (PPPoE). If you have unusual network requirements, check the OpenBSD website, mailing lists, or man pages to see if those requirements are supported.

Some datalink layers have been implemented over many different physical layers. Ethernet, for example, has been implemented over twinax, coax, CAT3, CAT5, CAT6, CAT7, optical fiber, and radio waves. And for true device independence, we have seen TCP/IP implemented with a biological transport layer: carrier pigeon.[1]

With minor changes to the device drivers, the datalink layer can address any sort of physical layer. This is one of the ways in which layers simplify the network.

Chapter 12 discusses Ethernet in detail, as it's the most common network type for OpenBSD systems. Once you understand how Ethernet works, you'll have no difficulty adding new datalink protocols as needed.

The datalink layer exchanges information with the physical layer and the network layer.

The Network Layer

The network layer is the part that maps connectivity between network nodes, answering questions like "Where are other hosts?" and "Can I get there from here?" This logical protocol provides a consistent interface to programs that run over the network, no matter what the physical and datalink layers look like.

The network layer used on the Internet is the Internet Protocol, or IP. Both version 4 (IPv4) and version 6 (IPv6) provide each host with one or more unique *IP addresses*, so that any other host on the network can find it. Okay, IPv4 network address translation munges the whole "unique address" rule, but your network still has a unique IP address somewhere.

---

1. You laugh, but the technical reviewer for this book was part of the first IP-over-carrier-pigeon implementation team that tackled the practical tests as specified in RFC 1149. That's how I knew he had the time to review this book in excruciating detail. (If that's how he spends his time, he couldn't very well claim he was too busy, now could he?)

The network layer talks to the datalink layer below it and the transport layer above it.

The Transport Layer

The transport layer is where actual data flows. The three most common transport layer protocols are the Internet Control Message Protocol (ICMP), Transmission Control Protocol (TCP), and User Datagram Protocol (UDP).

ICMP passes basic connectivity messages between hosts with IP addresses. If IP provides a road and addresses, ICMP provides traffic lights and highway exit signs. Most of the time, ICMP runs silently in the background.

UDP and TCP are the protocols that carry actual data between hosts, and they are so prevalent that the suite of Internet protocols is usually called TCP/IP. UDP is a bare-bones transport protocol, offering the minimum services needed to transfer data over the network. Its minimalism means that if you want to customize exactly how data flows in your application, you can build it out of valid UDP. TCP provides more sophisticated features, such as integrity checking and congestion control, but many of its settings are hard-coded.

In addition to these three, many other protocols run at the transport layer. The file */etc/protocols* contains a fairly comprehensive list of transport protocols built atop IP. While it lists many more protocols than you will ever actually deal with out in the wild, it doesn't include non-IP protocols like IPX/SPX or Digital Equipment Company's DECnet.

As an example, let's have a look at the first entry from */etc/protocols*:

```
ip      0       IP HOPOPT       # internet protocol, pseudo protocol number
```

Each */etc/protocols* entry has three key fields: an official name, a protocol number, and any aliases. The IP protocol, protocol 0, is known as IP and (*very* occasionally) as HOPOPT. Each protocol also has a comment giving it some context. Although some of the protocols in */etc/protocols* are long gone, some antediluvian devices out there might still speak them.

Note that ICMP, TCP, and UDP are slightly different when run over IPv4 versus IPv6. Each protocol has clearly defined fields in the IP packet header, leaving specific numbers of bits for things like checksums, destination addresses, and so on. You can't run a transport protocol over an incompatible network protocol—TCPv6 over IPv4 just doesn't work.

The transport layer speaks to the network layer below and to the applications layer above it.

Applications

Applications are definitely part of the network. Applications request network connectivity, send data over the network, receive data from the network, and process that data. Web browsers, email clients, JavaServer Pages (JSP) servers, and so on are examples of network-aware applications. Applications need to communicate with only the transport protocol and the user. The upper three layers of the OSI network model are inside applications.

Problems with the user layer are beyond the scope of this book, but I find that many of these issues can be solved with proper application of a large chainsaw.

The Life and Times of a Network Request

So how do all these layers fit together in the real world? Let's have a look at a hypothetical network request, and walk through how data traverses the layers and the network.

NOTE *Some of this discussion touches on topics covered later this chapter, so you might want to reread this section after finishing the chapter. Purists will notice that I skip a lot of parts of the process, but I'm trying to relay the basics of how TCP/IP works in practice, not model every painful detail of a real network transaction.*

Suppose a user connected to your network wants to look at a very important work-related website, such as Scott Meyer's Basic Instructions (*http://www.basicinstructions.net/*). The user opens his browser, enters the URL, and presses ENTER. The browser application transforms the user's request into the proper format and asks the transport layer for a TCP connection to a particular IP address on port 80.

The transport layer inside your computer examines the browser's request and allocates the appropriate resources for it. The request is broken up into digestible chunks, called *segments*, and handed down to the network layer.

The network layer doesn't care about the contents of the request; it's only concern is where that data is going. The network layer takes the TCP data and attaches the proper addressing information to it. The resulting chunk of data is called a *packet*. The network layer checks the packet's destination, chooses the interface closest to the gateway to that destination, and drops packets down into the datalink layer.

The datalink layer doesn't care about the contents of the packet, and it certainly doesn't care about IP addresses or routing. It has been given a lump of zeros and ones, and its job is to transmit those zeros and ones to another network node. The datalink layer adds the appropriate header and/or footer information to the packet, creating a *frame* appropriate for the physical layer. The frame's header and footer contain the addressing information for the physical layer. On most networks, the datalink layer prepares frames for the local Ethernet. Then the datalink layer hands off the frame to the physical layer for transmission.

The physical layer has no intelligence at all (think carrier pigeons). The datalink layer hands the physical layer a frame, and the physical layer transmits that frame to another physical device. For a web browsing client, this is usually the default router for the local Ethernet. The physical layer doesn't care about the upper-level protocols. Its only job is to make sure the frame gets to the destination without errors.

When the client computer's router receives the frame, it sends it up to the datalink layer. The datalink layer strips out the frame information and

hands the resulting packet up to the network layer. The router's network layer examines the packet, looks at its routing table, and decides which interface to send it out on. This might be another Ethernet interface, a T1, a DS3, an OC3, or whatever the router uses for upstream connectivity. Once the router chooses an interface, it hands the packet to the datalink layer for that interface.

The local router's upstream connection probably goes through a whole series of routers. Each router decides where to send the request based on its routing table. The request probably traverses a variety of datalink layers as it travels. Thanks to layering and abstraction, neither you nor your computer needs to know anything about any of them.

When the request reaches its destination, the computer at the other end of the transaction accepts the frame and sends it all the way back up the protocol stack. The frame is stripped down to packets, which are stripped down to segments, which are reassembled into a data stream. The data stream is then handed to the application (in this case, a web server). The application processes the request and returns an answer, which goes back down through the protocol stack and travels across the network, bouncing up and down through various datalink layers on the way as necessary.

This example shows why the layer model is important: Each layer knows only what it absolutely must about the layers above and below it, making it possible to swap out layers if necessary. When a new datalink protocol is created, the other layers don't need to change. The network layer just hands a packet to the datalink layer and lets the datalink do its thing. When you install a new network card, you need only a driver that interfaces with the datalink layer and the physical layer; you don't need to change anything higher in the network stack.

Network Stacks

A network stack is the software that lets a host communicate with the network. A host can run with an IPv4-only network stack, an IPv6-only network stack, or a dual-stacked setup.

You're already familiar with an IPv4-only stack—it's what most hosts ran for much of the past 30 years. An IPv4-only stack can communicate only over IPv4. Today, an IPv4-only stack gets you access to the entire Internet, with a few deliberate exceptions. That will not be true in a few years.

Likewise, an IPv6-only stack can communicate with only IPv6 hosts. Because most Internet sites don't yet support IPv6, running an IPv6-only stack isn't practical at this point. It is, however, an excellent way to test your IPv6 infrastructure and connectivity.

The most common configuration these days is a dual-stack setup. Client hosts try to use both IPv4 and IPv6, preferring one over the other. I recommend configuring hosts with dual stacks, preferring the stack with better connectivity. (If you get IPv6 connectivity through a tunnel, it's not as fast

as your IPv4 connectivity.) If you have equal IPv4 and IPv6 connectivity, use whichever you prefer. IPv6 works well enough that I often don't realize that I'm using it until I analyze my traffic.

You don't need to do anything special to enable IPv6 on OpenBSD—an IPv6 address, a default router, and a DNS server, and away you go.

IPv4 Addresses and Subnets

An *IP address* is a unique 32-bit number assigned to a specific network node. Some IP addresses are more or less permanent, such as those assigned to vital servers; others change as required, such as those used by desktop clients. Individual machines on a shared network use IP addresses from a range of addresses assigned to that network.

Rather than expressing that 32-bit address as a single number, an IP address is divided into four 8-bit numbers, usually expressed as decimals. While 192.0.2.1 and 11000000.00000000.00000010.00000001 represent the same address, the first option is easier for our feeble little brains to grasp.

Internet service providers (ISPs) issue IP addresses in blocks. These blocks are the smallest allocation that they can get away with giving you—say, 16 or 32 addresses. If your system is on a server farm, you might get only a few IP addresses out of a block of 256.

A *netmask* indicates the size of the block of IP addresses assigned to your local network. The size of your IP block determines your netmask—or, your netmask determines how many IP addresses the network has.

ISPs issue IP addresses by prefix length, commonly called a *slash*. You'll see IP address blocks described in forms like 192.0.2.128/26. Everyone who has worked with networking has seen the netmask 255.255.255.0, and most know that it's associated with a block of 256 IP addresses. That netmask is also called a /24. The number after the slash is the number of fixed bits in the netmask. Remember, an IPv4 address is a 32-bit number; on a /24 network, 24 of those bits will never change.

This isn't a textbook on binary math, so I won't quiz you on the conversions, but think of an IP address as a string of 32 binary digits. On your networks, you can change the bits on the far right, but not the bits on the far left. But where is the line that separates right from left?

Netmasks have traditionally been split on 8-bit boundaries, but there's no hard rule that says they must be. A /25 network has 25 fixed bits—one more fixed bit than what used to be called a class C network—leaving you with 7 bits to play with. The netmask's fixed bits are set to 1, and your network bits are set to 0, as in the following example of a /25 netmask:

```
11111111.11111111.11111111.10000000
```

The first three blocks are set to the binary 11111111, which is 255 in decimal. The last block is set to 1000000, which is 128. Mash these together, and your resulting netmask is 255.255.255.128.

If you reduce netmasks to binary, they're simple to figure out. While you won't need to work with this every day, if you don't understand the underlying concepts, the decimal conversion looks like total gibberish. With a little practice, you'll recognize certain decimal strings as legitimate netmasks.

So now that you know how netmasks work, what the heck does all this mean in the real world?

IP addresses are issued in multiples of 2. If you have 4 bits to play with, you have 16 addresses (2^4=16). If you have 8 bits to play with, you have 256 addresses (2^8=256). If someone says that you have exactly 17 IP addresses, you're either sharing a network with other people or they're wrong.

It's common to see a host's IP with the netmask attached, such as 192.0.2.130/26. This gives you everything you need to attach the host to the local network. (Finding the default gateway is a separate issue, but it's usually the top or bottom address in the block.)

Calculating a Decimal IPv4 Netmask

Converting from binary to decimal to binary is error-prone and mildly annoying. Here's how to calculate your netmask while remaining in decimal land.

Find how many IP addresses you have on your network. This will be a multiple of 2, almost certainly smaller than 256. Subtract the number of IP addresses you have from 256. This is the last number of your netmask. You still need to recognize legitimate network sizes, however. If your IP address is 192.0.2.251/26, you'll need to know that a /26 is 26 fixed bits, or 64 IP addresses. Your netmask is 255.255.255.192 (256–64=192).

And I should also mention that netmasks occasionally appear in hexadecimal.

Before you travel to my house to bludgeon me repeatedly with this book, Table 11-1 shows netmasks, IP information, and related information for /24 and smaller networks.

Table 11-1: IPv4 Netmasks and IP Address Conversions

Prefix	Binary Mask End	Decimal Mask	Hex Mask	Available IPs
/24	00000000	255.255.255.0	0xffffff00	256
/25	10000000	255.255.255.128	0xffffff80	128
/26	11000000	255.255.255.192	0xffffffc0	64
/27	11100000	255.255.255.224	0xffffffe0	32
/28	11110000	255.255.255.240	0xfffffff0	16
/29	11111000	255.255.255.248	0xfffffff8	8
/30	11111100	255.255.255.252	0xfffffffc	4
/31	11111110	255.255.255.254	0xfffffffe	2

When you don't feel like doing the math, you can refer to Table 11-1 or install the `ipcalc` package for quick netmask calculations. Don't say I never take pity on my readers.[2]

Viewing IPv4 Addresses

Display IP addresses with `ifconfig(8)`. If you run `ifconfig` without any arguments, it displays all interfaces on the machine.

```
$ ifconfig fxp0
...
        inet 192.0.2.226 netmask 0xfffffff0 broadcast 192.0.2.239
        inet 192.0.2.231 netmask 0xffffffff
...
```

The lines starting with `inet` are IPv6 addresses. This interface has the primary IPv4 address of 192.0.2.226 and a secondary, or *alias*, address of 192.0.2.231. You can also see the netmask of each of these addresses and the broadcast address for the subnet.

Unusable IPv4 Addresses

Every block of IPv4 addresses reserves the first and last IP addresses for use by the network:

- The first IP address in a block is the *network address*, used for separating networks (and on primordial BSD systems, the broadcast address). On a /24 network, this would be an address ending in .0.
- The last IP address in the block is the *broadcast address*. On a /24 network, the broadcast address ends in .255.

NOTE *According to the IP specifications, every machine on a network is supposed to respond to a request to the broadcast address. Unfortunately, in the late 1990s, this feature was used as an attack technique: All you needed to do was ping the broadcast address on any given network, and you would have a list of all IP addresses currently in use. Consequently, this functionality is now disabled by default on most operating systems and network appliances.*

You cannot assign the first or the last IP address in a network to a device without risking network problems. Some systems fail gracefully, others fail painfully, and a rare few make it work. Although OpenBSD won't object if you use the top and bottom network addresses, prepare for mayhem the first time you plug in a commodity printer or other embedded device. It takes only one inflexible device to ruin your whole day.

2. I never do take pity on my readers; I just don't want you to actually *say* so.

Special IPv4 Addresses

Quite a few blocks of IPv4 addresses are set aside for specific purposes. Although you don't need to know all of them, there are two groups you'll see pretty often. For a complete list of IPv4 subnets reserved for special purposes, read RFCs 5735 and 6598.

Localhost

The address range 127.0.0.1/8 is set aside for *localhost*, a machine's address for itself. Every Unix-like system—and most other operating systems—attaches 127.0.0.1/8 to a loopback interface. Everything knows that the localhost address is local to the specific machine. Packets to or from 127.0.0.0/8 should never cross the network; likewise, daemons bound only to 127.0.0.1 can be accessed on only the local machine.

Private Networks

Internet standard RFC 1918 sets aside three networks for use on private networks and behind network address translation (NAT) devices: 10.0.0.0/8, 172.16.0.0/12, and 192.168.0.0/16. While public IP addresses must be issued by an ISP, anyone can use addresses within these three blocks as long as those hosts are not directly exposed to the public Internet. If you have a network where hosts do not have access to the Internet, or if you provide Internet access through a proxy server or NAT, you can use an RFC 1918 network.

IPv4 Addressing Pitfalls

Common wisdom dictates that each computer on a network is assigned a single IP address for each of its network interfaces. One computer, one network card, one IP address—simple, right?

Not always. Some special-purpose interfaces (such as those dedicated to packet sniffing) function as intended without an IP address, and many operating systems will allow you to assign multiple IP addresses to a single network interface through a process called *aliasing*. You can also bond multiple physical cards into a single network interface, giving the computer one large virtual interface. While you might not deal with these configurations every day, keep them in mind when troubleshooting.

IPv6 Addresses and Subnets

There's a basic problem with IPv4: It provides only 4.29 billion addresses, and that's just not enough. Without subnetting, that's fewer than one address for every human being. Eventually, every person will have at least one IP-capable device.

Although IPv4 addresses haven't run out yet, they're becoming an increasingly scarce resource. Overly generous allocations in the early days, along with large chunks of address space reservations for special purposes, have accelerated exhaustion. The world is slowly grinding toward IPv4's replacement: IPv6.

Some parts of the world already use IPv6 extensively. Even if your network doesn't use IPv6 today, one day you'll need it—probably without warning. Prepare yourself now, or one day you'll discover that you needed it the week before.

IPv6 Basics

Like IPv4, IPv6 is a network layer protocol. IPv4 uses 32-bit addresses, usually expressed as four groups of decimal numbers from 0 to 255 (for example, 192.0.2.13). IPv6 uses 128-bit addresses, expressed as six groups of four hexadecimal characters separated by colons (for example, 2001:db8:0:bad:c0de:cafe). A 128-bit address space gives enough IPv6 addresses for every atom in the Earth to have more than 10 IP addresses. TCP, UDP, ICMP, and other protocols run atop it. IPv6 has its own layer 2 protocol, Neighbor Discovery, which replaces protocols such as Ethernet's ARP.

The good news is that you don't need to relearn the basics of networking. Hosts still need an IP address and a default gateway, routers still use a routing table, and you can almost—*almost*—substitute an IPv6 address for an IPv4 address and watch everything work. A web server doesn't care if it binds to port 80 on 192.0.2.13 or on 2001:db8:a12a:bad:c0de:café. The server just accepts requests sent to it and responds appropriately. That said, software does need to change slightly, because our web server must be able to log connections from both IPv4 and IPv6 addresses. These changes have wide-reaching repercussions, and we'll be sorting out edge cases for the next decade. But, in general, once you understand the new rules for IPv6, all of your networking knowledge is applicable.

Understanding IPv6 Addresses

As noted, IPv6 addresses are 128 bits, expressed as six colon-delimited groups of four hexadecimal characters each. As with decimal IPv4 addresses, you don't need to display leading zeros in each group. The address 2001:db8:0:bad:c0de:cafe could also be written as 2001:db8:0000:0bad:c0de:cafe, but just as we wouldn't write 192.000.002.013, we strip out the leading zeros in an IPv6 address.[3]

IPv6 addresses often contain long strings of zeros. This had to do with subnetting, which I'll describe later in this section. As of this writing, the IPv6 address of Sprint's website is 2600:0:0:0:0:aaaa. When consecutive groups include only zeros, as in this address, they're replaced with two colons (`::`). This IP address is usually displayed as 2600::aaaa. You can do this only once per address, however. You can't, for example, have the address 2600::1::1, because it's ambiguous. Does 2600::1::1 represent 2600:0:0:1:0:1 or does it represent 2600:0:1:0:0:1? I don't know, and neither does your server.

You've probably seen a port number added to an IPv4 address, such as 192.0.2.13:80. Using a colon to glue a port number to an IPv6 address

3. Some operating systems treat addresses containing numbers that begin with 0 as octal. Don't actually use addresses like 192.000.002.013, or you might get a base-8 surprise.

would be confusing. The IPv6 address 2001:db8::bad:c0de:cafe:80 isn't ambiguous, but if you read it quickly, you might miss the double colon and think this is an IP address ending in 80. If you're adding a port number to an IPv6 address, enclose the address in square brackets, as in [2001:db8::bad:c0de:cafe]:80.

Viewing IPv6 Addresses

Use `ifconfig(8)` to see all IPv6 addresses assigned on your machine. Here, I give `ifconfig` the name of my network card, `fxp0`.

```
$ ifconfig fxp0
...
        inet 192.0.2.13 netmask 0xffffff00 broadcast 198.0.2.255
        inet6 fe80::bad:c0de:cafe%fxp0 prefixlen 64 scopeid 0x2
        inet6 2001:db8::bad:c0de:cafe prefixlen 64 autoconf pltime 604399 vltime 2591599
...
```

The lines starting with `inet6` are my IPv6 addresses. This interface has been assigned two IPv6 addresses: `fe80::bad:c0de:cafe%fxp0` and `2001:db8::bad:c0de:cafe`. (Wait . . . where did that `%fxp0` come from? You'll find out in "Link Local Addresses" on page 195. For now, just nod and smile, and keep reading.)

IPv6 Subnets

Unlike IPv4, where you can subnet at any bit, IPv6 is usually subnetted at colon boundaries. Colons appear every 16 bits, so the natural IPv6 subnets are /16, /32, /48, and /64. Though IPv6 standards recommend using /64 as the smallest possible network, many carriers use /80, /96, and /112 networks for special purposes. (I've also seen people use subnets not divided at 16-bit boundaries. I won't cover them, but don't let your brain explode when you encounter a /51.) IPv6 subnets are always expressed as a slash, also known as a *prefix length*, so you won't see a netmask of ffff:ffff:ffff:ffff::, as you might in IPv4.

ISPs are usually issued a /32 or a /48 subnet and are expected to issue end-user networks, such as the typical home network, a /64 network. If ISPs do issue /64 subnets to their users, an end-user network will provide 2^{64} IP addresses, or 18,446,744,073,709,551,616 IP addresses. (This will suffice for any number of televisions, phones, refrigerators, water faucets, vacuums, and network-enabled tacos.)

When you subnet at 16-bit boundaries, each network has 65,536 subnets of the next smaller size. A /32 contains 65,536 /48 networks, and a /48 contains 65,536 /64 networks.

Special IPv6 Addresses

Like its predecessor, IPv6 reserves several blocks of addresses for special purposes. You don't need to memorize all of these reserved addresses, but some will appear in daily use.

localhost

IPv6's localhost address, ::1/128, works much like 127.0.0.1 in IPv4: It always refers to the local machine. In OpenBSD, ::1/128 is always assigned to the `lo0` interface.

Link Local Addresses

Addresses beginning with fe8*x*: (where *x* is variable) are local to their interface. Every link has such *link local* addresses that are valid only on a specific local network. Even if an IPv6 network has no router, the hosts on the local, directly attached network can find each other and communicate using these local addresses. These networks are always /64 subnets. You'll see identical IPv6 subnets on other interfaces and on networks completely disconnected from your network. That's okay. These addresses are local to the link. For example, here's a link local address on an OpenBSD machine:

```
inet6 fe80::bad:c0de:cafe%fxp0 prefixlen 64 scopeid 0x2
```

The link local address of this interface is `fe80::bad:code:cafe`. The trailing `%fxp0` indicates that this address is local to the interface `fxp0` and isn't usable on any other interface on the machine. If your machine has an interface `fxp1`, and a host on that network tries to reach the address fe80::bad:code:cafe, this machine will not respond. This particular address is valid only for the network attached to interface `fxp0`.

You might note that the link local address has a section in common with the public IPv6 address on this network. That's because an autoconfigured IPv6 address is usually calculated from the interface's physical address; it doesn't matter whether that autoconfigured address is on a public address or a link local address.

Assigning IPv6 Addresses

IPv6 clients can usually use autoconfiguration through *router discovery*, anIPv6 protocol where routers announce their presence on the network and the legitimate addresses to clients. Unfortunately, IPv6 autoconfiguration does not support common Dynamic Host Configuration Protocol (DHCP) options, such as assigning a Domain Name Service (DNS) server, let alone the options used for diskless configuration. If you have configured a DNS server—even IPv4 servers accessible on a dual-stacked host—autoconfiguration works just fine. If you run an IPv6-only network, you must either set up an IPv6 DHCP server to provide DNS server information to clients or configure DNS servers manually.

Servers should not use IPv6 autoconfiguration. A server usually needs a static IP address, even in IPv6. Similarly, routers cannot use autoconfiguration. If a host can forward packets, it requires a static IPv6 address.

You can assign multiple IPv6 addresses to a single interface by using aliases, just as with IPv4.

In IPv6, a client on a /64 network can use autoconfiguration.

IPv6 autoconfiguration resembles a stripped-down DHCP service. The router broadcasts subnet and gateway information, and the hosts configure themselves to use it. Hosts on a network smaller than /64 must be manually configured.

Remedial TCP/IP

Now that you have a simplified overview of how the IP system works, let's look at a real network protocol in some depth. The dominant transport protocol on the Internet is the Transmission Control Protocol over Internet Protocol, or TCP/IP. Although TCP is a transport protocol and IP is a network protocol, the two are so tightly intertwined that they're generally referred to as a single entity.

We'll start with ICMP, and proceed to UDP and TCP.

ICMP

ICMP is used to transmit routing and availability messages across the network. Tools such as `ping(8)` and `traceroute(8)` use ICMP. ICMP includes all sorts of different protocols and tools.

While some people claim that you need to block ICMP for security purposes, those people don't understand that ICMP is just as diverse as the better-understood transport protocols TCP and UDP. Proper IPv4 network performance requires large chunks of ICMPv4. If you must block ICMPv4 for security reasons, do so selectively. For example, blocking source quench messages breaks path maximum transmission unit (MTU) discovery, which will steer you directly into a world of hurt. If you don't understand that last sentence, don't block ICMPv4.

IPv6 dies without ICMPv6, as IPv6 doesn't support packet fragmentation, so never block ICMPv6. If you don't know what packet fragmentation is, just trust me on this.[4]

UDP

UDP is the most bare-bones data-transfer protocol that runs over IP. It offers no error handling, minimal integrity verification, and no defense whatsoever against data loss. The transport protocol considers each packet of UDP completely self-contained; there are no data-coherence checks at the protocol layer. Despite these drawbacks, UDP can be a good choice for particular sorts of data transfer, and many vital Internet services rely on it.

NOTE *This discussion covers both UDPv4 and UDPv6. While each runs over only the corresponding network protocol, they behave identically otherwise.*

UDP is also a datagram protocol, meaning that each network transmission is complete and self-contained, and received as a single integral

4. Or you can go look it up. Whatever—you not believing me won't hurt my feelings.

unit. While the application might not consider a single UDP packet a complete request, the network does.

When a host transmits data via UDP, it has no way of knowing if the data ever reaches its destination. Programs that receive UDP data just listen to the network and accept whatever happens to arrive. When a program receives data via UDP, it cannot verify the source of that data. Although each UDP packet does include a source address, this address is easily faked. Each UDP packet includes a checksum for the packet, but there's no integrity checking for the data stream as a whole. This is why UDP is called *connectionless*, or *stateless*.

No integrity checking, no guard against data loss, the potential for faked packets—all this sounds pretty unreliable. So why use UDP at all?

UDP-based applications often have their own error-correction methods or otherwise don't mesh well with more reliable protocols, such as TCP. For example, simple client DNS queries must time out within just a few seconds or users will whine uncontrollably. TCP connections time out only after two minutes. DNS requires quick failures and only a single packet per transaction, which makes UDP a better choice than TCP for simple DNS queries. Real-time streaming services, such as video conferencing applications, also use UDP. (After all, if a few pixels go missing during a video conference, you don't want those pixels a minute later.) Most other UDP-based applications use UDP for similar reasons.

Because the UDP protocol itself doesn't return anything when you connect to a port, there's no reliable way to remotely test if a UDP port is reachable (although tools such as `nmap` try to do so).

If you want a protocol that responds at the network layer, look at TCP.

TCP

TCP includes nifty features, such as error correction and recovery. The receiver must acknowledge every packet it gets; otherwise, the sender retransmits any unacknowledged packets. Unlike UDP, applications that use TCP can expect reliable data transmission. This makes TCP a *connected*, or *stateful*, protocol.

NOTE *This discussion covers both TCPv6 and TCPv4. While they differ because of their underlying transport protocol, they behave in the same way.*

TCP is also a *streaming protocol*, which means that a single request can be split among several network packets. While the sender might transmit several chunks of data one after the other, that data might arrive out of order or fragmented. The recipient must track these chunks and assemble them properly to complete the network transaction.

For hosts to exchange TCP data, they must set up a channel for that data to flow across. One host requests a connection, the other host responds to the request, and then the first host starts transmitting. This setup process is known as the *three-way handshake*. Similarly, once transmission is complete, the systems must do a certain amount of work to tear down the connections.

To test if a TCP port is open, you can use `telnet(1)` or `nc(1)` to connect to the port. Here, I see if I can connect to port 22 on the host `caddis`.

```
$ telnet caddis 22
Trying 192.0.2.35...
Connected to caddis.
Escape character is '^]'.
SSH-2.0-OpenSSH_6.0
❶ ^]
❷ telnet> c
Connection closed.
```

I connect to the remote port and see information displayed by the port, use the telnet escape character **^]** (CTRL-]) to disconnect ❶, and enter **c** ❷ to close telnet.

TCP is commonly used by applications most suited to its fairly generic set of timeouts and transmission features, such as email programs, FTP clients, and web browsers.

How Protocols Fit Together

You can compare the network stack to sitting with your family at a holiday dinner. The datalink layer (ARP, in the case of Ethernet) lets you see everyone else at the table. IP gives every person at the table their own unique chair (except for the twins using piano bench NAT). ICMP provides basic, lower-layer information such as "The quickest way to the baked sweet potatoes is to get Uncle Mike to pass them"[5] or "Aunt Liz can't lift the ham platter." TCP is where you hand someone the butter and the other person must say "thanks" before you let it go. UDP is like tossing a roll at Grandma Lucas; she might catch it or it might bounce off her forehead.

Transport Protocol Ports

Transport protocol ports permit one server to serve many different services over a single transport protocol, multiplexing connections between machines. When a network server starts, it attaches, or *binds*, to one or more logical ports. A logical port is just an arbitrary number ranging from 0 to 65536, although nothing uses port 0. For example, Internet mail servers often bind to port 25.

Each TCP or UDP packet arriving at a system carries a field containing its desired destination port number. If an incoming packet asks for port 25, it is connected to the mail server running on that port. This means that other programs can run on other ports, clients can talk to those different ports, and no one gets confused except you.

Note that port assignments are not some sort of physical constant, but rather are mutually agreed upon. There's no reason that email services should run on port 25 other than the fact that everyone agrees that they

5. For the record, Uncle Mike's security policy prevents him from passing baked sweet potatoes. If you want them, you're going to have to take them by force.

should. If someone tries to send you email, their mail server will automatically connect to port 25 on your server. If you run email on port 80 and have a web server on port 25, you'll never get your email, and your web server won't get much traffic.

The file */etc/services* contains a list of port numbers and the associated services. The file has a very simple, five-column format, as shown in these two sample lines:

```
www             80/tcp          http    # WorldWideWeb
www             80/udp                  # HyperText Transfer Protocol
```

The first field is the name of the service assigned to this port. This entry is for the service `www`. Port 80 is assigned to `www`, both TCP and UDP. Then there's a list of any other names assigned to this port. Port 80 is also known as `http`. Finally, there's a comment that gives more detail about the service.

The HTTP protocol used on the Web runs over TCP, so why is UDP port 80 also reserved for HTTP? The answer is pretty simple: Computer people are easily confused. Having two services share the same port number but run on different protocols confuses people—for example, the `syslog` service runs on port 514 via UDP, and the `lpr` printer protocol runs on port 514 over TCP.[6]

Some server programs read */etc/services* to learn which port to bind to on startup, and many client programs read */etc/services* to learn which port they should try to connect to. If you run servers on unusual ports, you might need to edit this file to get these programs to attach where needed.

As with all standards, there are times you will want to break the rules. The SSH daemon `sshd` normally binds to port 22/TCP, but I've run it on ports 23 (`telnet`), 80 (`www`), 443 (`https`), and others to evade naïve packet-filtering firewalls. You will find your own reasons to break the standards. That's fine, as long as you understand what you're doing and how it affects others.

Reserved Ports

Ports below 1024 in both TCP and UDP can be opened only by the root user. These ports are assigned (mostly) to core Internet infrastructure protocols, such as DNS, SSH, HTTP, LDAP, and so on—services that only a few select hosts on each network should offer. Only programs with root-level privileges can bind to reserved ports. For example, a user can run a game server on a high-numbered port if the system policy allows, but that's different from setting up a web page visible to the whole world that claims the machine's official purpose is a game server. The port assignment for these core protocols is generally permanent, and if you want to interoperate with other sites, you won't change them.

6. I used to count how many people confused 514/tcp and 514/udp, but the number got so high that I got depressed, so I stopped.

OpenBSD software usually binds to a reserved port as root and then drops privileges, performing the rest of its functions as an unprivileged user. These unprivileged users, discussed in Chapter 6, have even fewer privileges than a normal user account.

If you must run a service that binds to a reserved port, and it can run only as root, consider carefully whether you actually need it. Try to find an alternative server that does privilege separation. If you can't, at least install that service on a dedicated machine to reduce its threat to other services on your network.

Which Ports Are Open?

So, network services are made available via TCP or UDP ports. Programs bind to ports to offer network services. This brings up two obvious questions:

- Which ports are open?
- What programs are listening to each port?

You can answer these questions with `netstat(1)` and `fstat(1)`.

Using netstat

The `netstat(1)` program provides general visibility into the network stack. Use `netstat` to check your routing table, examine open sockets, see how many packets are traversing your interfaces, and so on. (I could write an entire book about `netstat`, but no one would buy it. Instead, I'll sprinkle bits of `netstat` magic throughout this book.)

When looking at network information, I recommend turning off DNS lookups by using the `-n` flag. You can always rerun a check with DNS turned on, but adding DNS queries to the network sockets can sometimes skew the information you're viewing, and almost always slows the command.

The `-f` argument lets you choose a protocol family to display. Use `-f inet` to see only IPv4 sockets, or `-f inet6` to see only IPv6. Read `netstat(1)` for the full protocol list.

Finally, `-a` tells `netstat` to show all sockets opened by any process, rather than just sockets owned by the user.

Let's put all those options together and have a look at the output. Here, I show the open IPv4 sockets on my system:

```
$ netstat -na -f inet
Active Internet connections (including servers)
Proto   Recv-Q Send-Q    Local Address         Foreign Address     (state)
❶tcp         ❷0      0  ❸192.0.2.135.22       ❹192.0.2.8.49997    ❺ESTABLISHED
tcp          0      0   127.0.0.1.587         ❻*.*                ❼LISTEN
tcp          0      0   127.0.0.1.25          *.*                  LISTEN
tcp          0      0  ❽*.22                  *.*                  LISTEN
Active Internet connections (including servers)
Proto   Recv-Q Send-Q    Local Address         Foreign Address     (state)
❾udp         0      0   127.0.0.1.512         *.*
udp          0      0   *.514                 *.*
```

The list starts with open TCP ports ❶. The `Recv-Q` and `Send-Q` columns ❷ show the number of bytes that the system is in the process of receiving or trying to send.

The `Local Address` column shows the IP address attached to the local machine where this socket is listening. It's possible—common, even—for a service to bind to a port on only a single address on a machine. If the port is part of an actual connection, as the first example ❸ shows, the IP address is followed by the port number. This particular TCP connection is attached to port 22 at the address 192.0.2.135. Port 22 is reserved for SSH, so this is probably an SSH connection.

If the local address is an asterisk followed by a port number ❽, this is a wildcard bind. A program has bound to this port, and has asked the kernel to figure out the IP address. It's probably (but not necessarily) a listening socket.

The `Foreign Address` column ❹ shows the IP address and port of the remote host involved in a connection. If there's a foreign address shown, it always includes the port. If this column shows two asterisks ❻, that means the service is waiting for a connection on the local port.

The (`state`) column applies only to TCP connections. A live and working TCP connection is in the `ESTABLISHED` state ❺. Other states (`SYN_RCVD`, `ACK`, and `SYN+ACK`) are all normal parts of connection creation, while `LAST_ACK`, `FIN_WAIT_1`, and `FIN_WAIT_2` mean that the connection is closing. A state of `LISTEN` ❼ means that this socket is waiting for an incoming connection.

UDP ports are given their own section ❾. You might see remote hosts in the UDP section, especially for long-running protocols such as NFS and NTP, but remember that UDP is stateless, so you'll never see state on a UDP connection.

If you're interested in only TCP or UDP sockets, you can use the `-p` flag to show only a particular protocol. Here, I look at TCP sockets:

```
$ netstat -na -p tcp
Active Internet connections (including servers)
Proto   Recv-Q Send-Q  Local Address          Foreign Address      (state)
tcp          0     52  192.0.2.135.22         192.0.2.8.49997      ESTABLISHED
tcp6         0      0  ::1.587                *.*                  LISTEN
tcp          0      0  127.0.0.1.587          *.*                  LISTEN
tcp6         0      0  ::1.25                 *.*                  LISTEN
tcp          0      0  127.0.0.1.25           *.*                  LISTEN
tcp          0      0  *.22                   *.*                  LISTEN
tcp6         0      0  *.22                   *.*                  LISTEN
```

While this looks similar to the first output example, note that we see both IPv4 and IPv6 TCP connections and services. TCP runs over both IPv4 and IPv6, so choosing it shows both address families. It's entirely possible to have a service running on one address family and not the other. Many of my systems listen for incoming SSH connections only on IPv6; doing so hides me from port scanners and worms (for now, anyway).

Rather than listing every service waiting for an incoming connection, you can show only live connections by dropping the -a flag:

```
$ netstat -np tcp
Active Internet connections
Proto   Recv-Q Send-Q  Local Address       Foreign Address    (state)
tcp          0     52  192.0.2.135.22      192.0.2.8.49997    ESTABLISHED
```

Using fstat

Now that you know which TCP and UDP ports are open, how can you tell which programs are listening on them? OpenBSD includes `fstat(1)`, a program that displays all open files and sockets on the system. Network connections are open sockets. Running `fstat` on an idle system can generate hundreds of lines of output—one entry for each file opened by any process. While that's educational and useful, it's not what we're looking for. Specifically, we want to see which programs are holding network sockets open. The string `internet` indicates network sockets.

```
$ fstat | grep internet
mwlucas  sshd       21403    3* internet stream tcp 0xd5365994 192.0.2.235:22 <-- 192.0.2.8:49997
root     sendmail   19063    4* internet stream tcp 0xd537e330 127.0.0.1:25
root     sendmail   19063    5* internet6 stream tcp 0xd537e4c8 [::1]:25
root     sendmail   19063    6* internet stream tcp 0xd537e660 127.0.0.1:587
root     sendmail   19063    7* internet6 stream tcp 0xd537e7f8 [::1]:587
root     sshd       29561    3* internet6 stream tcp 0xd537e000 *:22
root     sshd       29561    4* internet stream tcp 0xd537e198 *:22
_syslogd syslogd    12885    4* internet dgram udp *:514
```

First, you see an `sshd` process owned by the user `mwlucas`. That's an unprivileged process, tied to a particular SSH session. Further down the list, you see an SSH daemon owned by root listening to the network. When a connection request arrives, the root-owned SSH daemon will hand it off to an unprivileged child process. You can also see that we have a variety of `sendmail` processes listening to the network.

This system runs the expected SSH and email servers, and no one has bound anything to odd ports. My nasty paranoid suspicions were unfounded (this time, anyway).

Between `netstat` and `fstat`, you should be able to get a good idea of what your system is doing on your network at any given time.

IP Routing

Most sysadmins don't need to understand much about IP routing, because most servers have only one network interface and one default gateway. The network administrator gives you an IPv4 address and a default route, you put them in the appropriate configuration files, and you're routed.

You don't need even that for most IPv6 hosts, as autoconfiguration makes things magically work. Servers will need a static IPv6 address and a manual default route.

Some servers have multiple interfaces, such as one to their default gateway and another to a group of related application or backup servers. OpenBSD systems frequently wind up in the network infrastructure, however, or sit in demilitarized zones (DMZs) where the server must make routing decisions. If you want to use OpenBSD in such an environment, or as a firewall, you must understand the basics of routing.

Routing is simply deciding where to send packets. If your system is attached to a network, it doesn't need to make any decisions; it just sends the packet to that network. Your system on 192.0.2.0/24 already knows how to reach any IP address beginning with 192.0.2—it can just send everything out to the local Ethernet. Where does it send those packets?

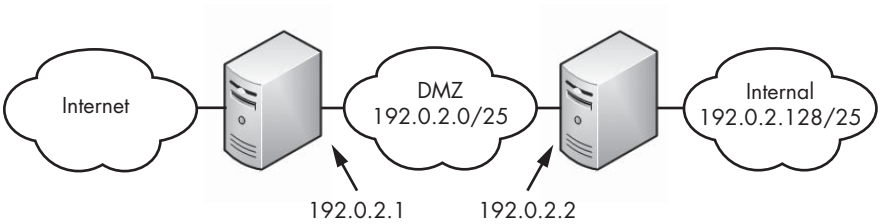
Most computers use a *default route*, which is an IP address on the local network where they send all packets bound for nonlocal IP addresses. This is very common where one router or firewall provides all network access. This device probably also has a default route that points to your ISP, which makes all the complicated routing decisions for you.

In other cases, you might have a dynamic routing protocol running on your network. If you're using Open Shortest Path First (OSPF), Border Gateway Protocol (BGP), or Routing Information Protocol (RIP), OpenBSD has daemons specifically for integrating these protocols. There's an introductory middle ground between full dynamic routing and simple default routes, however, and you should understand it before attempting full dynamic routing.

We'll cover a simple case here using an IPv4 example. (IPv6 routing is exactly like IPv4 routing, but with a lot more colons.)

IPv4 Routed Network Example

If a network has multiple gateways leading to different networks, hosts on the network must make routing decisions. Suppose your network has multiple routers attached to it, each going to a different network. Machines on your network decide where to send packets. Here's an example of a common double-firewall situation:



In this network design, hosts must transit a DMZ before entering either the Internet or the internal network. (Other designs exist, such as the hub-and-spoke model, but I've specifically chosen a design where routing is needed.)

The external firewall provides one layer of protection. It permits only traffic specifically deemed necessary (we'll go into the default deny stance in Chapter 21). It does, however, permit incoming connections to hosts in your DMZ.

The hosts in the DMZ are to some extent vulnerable. They are not trusted enough to be on the internal network. Your intrusion-detection systems or your web servers might live here.

The internal firewall, like the external firewall, permits only traffic deemed necessary to organization purposes. It probably doesn't allow any connections from the outside world, however, and it doesn't trust the hosts on the DMZ.

Only highly trusted hosts are permitted on the internal network. This is where the organization keeps its most precious data, such as the financial records, customer databases, and movie collections.

Many of the hosts in this network need to make only very simple routing decisions. Anything on the internal network has just one way to reach anything, and any host on the Internet has only one way to reach the internal or DMZ networks.

The external firewall is directly attached to the DMZ network, so it can send packets to those hosts. It needs a default route pointing to the Internet so it can reach the rest of the world. To reach the hosts on the internal network, it must send packets to the internal firewall's external interface. If you don't configure this on the external firewall, data will never reach the internal firewall. Because the external firewall is responsible for the internal network's Internet access, losing this route would disconnect the internal network from the Internet; internal systems could send packets, but would never receive any. The external firewall needs routing.

Similarly, you could configure routing on each host inside the DMZ. In that case, ICMP redirects from the firewalls would provide routing for these hosts, but trusting ICMP redirects on a vulnerable network is unwise and messy because it assumes that every host on the DMZ and every firewall accepts and sends ICMP redirects. If you're using OpenBSD, you want your server to be secure, so configure routing on your DMZ systems.

In this example, I configure routing for the external firewall. Configuring routing for the DMZ hosts is nearly identical to this example.

Managing Routing with route(8)

The `route(8)` command manages all system routing. Like `netstat`, `route` has several subfunctions that allow you to view, edit, and monitor the system routing table. While the `route(8)` man page has complete details, the ability to view, add, and delete routes should be enough to get you started.

Viewing Routes

OpenBSD, like any other network device, keeps routes in a routing table. To view the IPv4 and IPv6 routes, enter **`route show`**. Add **`-n`** to remove IP-address-to-name translations.

Here's the IPv4 routing table:

```
$ route -n show
Routing tables

Internet:
Destination        Gateway            Flags   Refs      Use   Mtu  Prio Iface
❶ default            192.0.2.1          UGS        4     6414     -     8 vic0
❷ 127/8              127.0.0.1          UGRS       0        0 33196     8 lo0
❸ 127.0.0.1          127.0.0.1          UH         1      170 33196     4 lo0
❹ 192.0.2.32/24      link#1             UC         1        0     -     4 vic0
❺ 192.0.2.1          00:0c:42:20:7f:42  UHLc       1        0     -     4 vic0
❻ 224/4              127.0.0.1          URS        0        0 33196     8 lo0
```

The table shows the following information:

- The `Destination` field lists the range of IP addresses this route applies to—destination addresses. The `default` entry indicates the default gateway, which is where the system sends all packets that have no specific route.
- The `Gateway` field tells where packets for this route should be sent. A gateway could be a hostname, an IP address, or a network interface.
- The `Flags` field contains markers that indicate what sort of route this is and how it behaves. The next section covers the various route flags.
- The `Refs` field shows the number of references to the route in the kernel (also known as the *refcounter*). If the refcounter drops to zero, the route is removed. This has no practical use for system administration, because one reference is sufficient to keep the route in the routing table; additional references don't change anything.
- The `Use` counter increments each time a packet uses that route.
- The `Mtu` is the MTU—the largest frame size that can travel over this route. If the field contains a hyphen (`-`), OpenBSD uses the MTU of the underlying physical interface. The loopback interface, `lo0`, isn't a physical interface, so OpenBSD explicitly sets the MTU very high. You might see a route with a lower MTU if Path MTU Discovery has kicked in.
- The `Prio` field gives the route priority. OpenBSD supports multiple routes to a single destination. Some routes are more desirable than others, and OpenBSD will use the route with the lowest priority number. Routes provided by dynamic routing protocols, such as BGP or OSPF, get higher priority numbers than static routes.
- The `Iface` field shows which interface this route uses.

NOTE *OpenBSD also includes dynamic routing daemons such as `ospfd(8)` and `bgpd(8)`. I don't cover them here, because that topic would fill a book on its own.*

Let's see what's interesting in the routes in this sample. The first entry at ❶ is the system default route. If there is no more specific route, packets will be sent to the IP address 192.0.2.1.

To reach the network 127.0.0.0/8 at ❷, packets should go to the IP address 127.0.0.1. 127.0.0.0/8 is the address range reserved for loopback addresses, and 127.0.0.1 is always the local machine. Notice the high MTU; this is a software interface, so there's no physical limit on the size of frames sent through it.

To reach the IP address 127.0.0.1 at ❸, send the packets to the IP address 127.0.0.1. This might seem a bit pedantic, but it's a valid route and needs to be in the table. Remember that 127.0.0.1 is always the loopback address of the local machine.

To reach the IP address 192.0.2.0/24 at ❹, use a gateway of `link#1`. This is a local physical interface—in this case, our Ethernet interface. The interface named `link#1` is actually the interface with index `#1`, which isn't really exposed to the system administrator anywhere else. These addresses are local to the machine, and you must figure out which interface this is by the IP address attached to the machine. Addresses local to the machine don't actually need to be in the routing table, but no one has bothered to remove this historical nit.

To reach a specific IP address on the local network at ❺, you'll get a route of the IP address and the physical media address. Because this host is connected via Ethernet, the gateway is a MAC address. Every local address that the system needs to find gets a route entry, and you should almost always show a specific route for the default gateway.

The last route at ❻ is for the multicast address range 224/8. If you're not using multicast, it should go to the local host.

NOTE *Multicast is a complicated topic beyond the scope of this book (again). But if you're interested, OpenBSD supports multicast just fine.*

Route Flags

The `Flags` column of the routing table indicates how routes are generated or used. `netstat(1)` contains a complete list of route flags. Table 11-2 lists the common ones.

Table 11-2: Common Route Flags

Flag	Description
`C`	This route was cloned.
`c`	This is a protocol-specific route (such as to an Ethernet MAC address).
`D`	This route is dynamic.
`G`	This route goes via a gateway.
`H`	This route is for a specific host.
`L`	This route is for the local link layer.
`M`	This route was modified.
`R`	This is a reject route. Packets are dropped, and an error is sent.
`B`	This is a blackhole route. Packets are dropped silently.

These flags tell you where a route came from and how it's used.

Adding Routes

Add routes with the `route add` command. You must know the destination network, its netmask, and the gateway.

```
# route add address-block/netmask netmask gateway
```

In our example network, the outer firewall needs a route to reach the private network, 192.0.2.128/25. To route this network to the inner firewall at 192.0.2.2, run this command:

```
# route add -net 192.0.2.128/25 192.0.2.2
add net 192.0.2.128: gateway 192.0.2.2
```

Packets will use that route immediately. If you run `route show`, you'll see that new route.

To add a default route, run `route add default` with the IP address of the default gateway, like this:

```
# route add default 192.0.2.1
add net default: gateway 192.0.2.1
```

To add routes automatically at boot, put the `route` statement in the */etc/hostname.if* file that leads to the destination network. These routes appear when the interface is brought up, before */etc/rc.securelevel* runs or any local daemons start. You'll see examples of using *hostname.if* for routes in the next chapter.

To add a default route automatically at boot, put the default router IP address in */etc/mygate*.

Deleting Routes

To delete a routing table entry, use `route delete` with the network address and netmask.

```
# route delete address-block -netmask netmask
```

To remove the route added in the previous example, run this command:

```
# route delete -net 192.0.2.128 -netmask 255.255.255.128
delete net 192.0.2.128
```

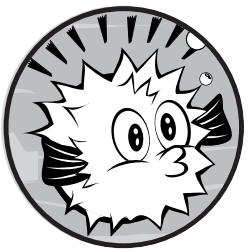
You should now have a decent idea of how routing works.

Now that you know how things are supposed to fit together, let's see how to configure Ethernet networks.

12

CONNECTING TO THE NETWORK

My tunnel is now up.
I can do IPv6.
Me and three others.



All that theory of IPv4 and IPv6 is fine. Now let's do something with it, and actually connect to a network. While dial-up connections work with OpenBSD, they're not used much these days, so we'll focus on Ethernet connections. Ethernet is the most common network type today, and the most common network interface on OpenBSD systems.

Most people have IPv4 connectivity, but IPv6 is increasingly important. If you can't get native IPv6 to your network, you can use a tunnel to reach IPv6 address space and provide IPv6 to your clients. I'll cover acquiring and configuring such a tunnel in this chapter.

Finally, OpenBSD can combine network connections into trunks or split them into virtual local area networks (VLANs). This chapter covers both approaches.

DNS Resolution

You'll probably want to use hostnames rather than IP addresses, so that you'll be able to browse to *http://www.cnn.com/* instead of *http://157.166.255.18*. Unix-like systems use the *resolver* to accomplish this feat.

Most hosts use two tools to map between IP addresses and hostnames: the *hosts* file and DNS. (Different operating systems support additional name services, such as YP, LDAP, NIS, and so on, but dang near every system supports these two.)

The *hosts* file is a text file on the local machine that contains static IP address and hostname lists. DNS is a more dynamic service that reaches across the network to find information. You can specify DNS servers by IP address, but we'll look at the *hosts* file in a little more detail.

What if you prefer IPv4 or IPv6 addresses? Or you want the *hosts* file to override DNS? Maybe you have a default domain that your queries should use. The resolver searches until it either finds the first answer or has exhausted its information sources, so these questions matter. Tell your resolver your needs in */etc/resolv.conf*.

The /etc/resolv.conf File

You configure the resolver behavior in */etc/resolv.conf*. A system without */etc/resolv.conf* can find only hostnames listed in the *hosts* file. Because the *hosts* file starts off empty, that's probably not what you want. Start by specifying domain names.

Default Search Domains

If you wanted to ping a host on a remote network, you might expect to need to specify the whole domain. Entering **`ping www.openbsd.org`** should work. But if you wanted to ping your company's web server, it would make more sense to just type **`ping www`**. And you can, because OpenBSD allows you to specify default domains, so that when you type in a short hostname, it will try to find the proper host.

For example, if you have only one local domain, you would list the domain keyword in */etc/resolv.conf* like this:

```
domain michaelwlucas.com
```

Now, when I enter `ping ftp`, the resolver should get the IP address of the host *ftp.michaelwlucas.com*.

If you have more than one local domain, use the `search` keyword and a list of domains, like this:

```
search michaelwlucas.com openbsd.org
```

If I enter `ping ftp` now, the resolver should get the IP of the host *ftp.michaelwlucas.com*. Once the resolver learns that no such host exists,

it will check for *ftp.openbsd.org*. Because that host exists, `ping` will start to work. The `search` keyword can have up to six domains, and can be no longer than 1024 characters.

Using Domain and Search

You can only use either `domain` or `search`. If you use both, the last entry in the file wins. If you list multiple search or domain lines, the last one in the file takes effect. Here's how not to do it:

```
search cnn.com openbsd.org
search sluggy.com michaelwlucas.com
domain blackhelicopters.org
```

You might as well get rid of the two `search` statements. The resolver will never go through those domain lists; it will use only the `domain` list because it's the last one.

Name Servers

Now that the resolver knows which domains to check by default, tell it which name servers to use. List each name server on its own line, by IP address, in order of preference.

```
nameserver 192.0.2.5
nameserver 198.51.100.5
nameserver 2001:db8::5
```

You can list up to three name servers, by IP address. (Hostnames in a `nameserver` entry won't work, for fairly obvious reasons.)

If your *resolv.conf* doesn't list a name server, the resolver should check for a name server on the local machine.

Lookup Order

You might get host information from DNS or from the *hosts* file. The resolver should stop once it finds an answer to a query. If you check the *hosts* file and then DNS, entries in the *hosts* file override the name server. If you check the name server before the *hosts* file, the *hosts* file is used only when no DNS record is available. Either approach has its uses, but by default, the resolver checks the *hosts* file, and then checks DNS. To reverse this, use the `lookup` keyword.

```
lookup bind file
```

The `file` option represents */etc/hosts*, while due to a historical accident, `bind` represents DNS. (The first DNS server software was the Berkeley Internet Name Domain server, or BIND.)The reverse (`file bind`) is the default, so there's no need to explicitly specify it.

Preferred IP Protocol

The resolver defaults to searching for IPv4 records first, and then looking for IPv6 records. To reverse this, use the `family` keyword.

```
family inet6 inet4
```

Again, the reverse is the default, so there's no need to use this keyword in that case.

The /etc/hosts File

The */etc/hosts* file matches IP addresses to hostnames. While the *hosts* file is very simple, its contents are available only on the local machine. A *hosts* file is most useful on a small private network, such as in your home or test lab. You can also use a *hosts* file to override data from the DNS server, such as when you want to test a new system.

Each line in */etc/hosts* represents one host. The first entry on each line is an IP address. The second is the fully qualified domain name of the host. Following these two entries, you can have an arbitrary number of aliases for that host. I often add comments at the end of the line, prefixed with a hash mark (#).

There was a time when I had a small network at home with only four machines: the proxy/firewall, the wife's desktop, my laptop, and the crash machine where I did stupid things. The *hosts* file looked like this:

```
192.0.2.1 ❶nat.blackhelicopters.org     ❷nat firewall gateway
192.0.2.8   boss.blackhelicopters.org     boss wife  ❸#don't crash
192.0.2.20  crashbox.blackhelicopters.org crashbox test
192.0.10.21 laptop.blackhelicopters.org   laptop mwlucas
```

The machine `nat.blackhelicopters.org` at ❶ also had the names `firewall` and `gateway` at ❷. I added a note to remind myself at ❸ not to run security scanners against my wife's desktop. (The machine `crashbox` is also called `test`.)

Any machine with this *hosts* table could find any machine listed in the *hosts* table by name. For example, I could run `ping boss` or `ssh crashbox` and reach the desired machine.

The *hosts* file works just fine for finding networked hosts, but whenever you add, remove, or change a machine, you must edit */etc/hosts* on every computer. And every time you change an IP address, you must edit */etc/hosts* on every machine.

NOTE *Unfortunately,* /etc/hosts *does not scale. When I got a fifth machine, I added an internal-only DNS server and emptied the* hosts *file on all of my systems.*

Resolver vs. Dynamic Configuration

If your OpenBSD system roams between networks, like a laptop, you probably use DHCP to configure your network connection.

DHCP overwrites */etc/resolv.conf* with the information for its network. This is appropriate for most users, but if you're carrying an OpenBSD laptop, you're not normal. You probably want some of your resolver configuration, such as your domain search list, to remain in effect no matter what network you're on.

OpenBSD supports permanent resolver configuration in the file */etc/resolv.conf.tail*. When OpenBSD's DHCP client gets */etc/resolv.conf* information from the server, it writes to */etc/resolv.conf* and adds `/etc/resolv.conf.tail` to the end.

Remember how only the last `search` or `domain` keyword works? *resolv.conf.tail* takes advantage of that, allowing you to override your network administrator's search order.

Ethernet

Ethernet is a shared network, meaning that many different machines can connect to the same Ethernet and can communicate directly with each other. I'm going to assume that you're using Ethernet as found in an average office or datacenter. Also, although Ethernet has been implemented over many different physical media, I'll assume you're working with CAT5 or better cable—today's most popular choice. If you use some unusual media type, or your card supports multiple media, you might need to manually set your preferred media on your interface.

Protocol and Hardware

Ethernet is a *broadcast protocol*, which means that every packet you transmit can be sent to every host on the network (although most Ethernet hardware limits recipients). Either your network card or your device driver separates the data intended for your computer from the data meant for other computers. A section of Ethernet where all hosts can communicate directly with all other hosts, without involving a router, is called a *collision domain* or *segment*.

You connect Ethernet segments with *hubs*, which are hardware items that can physically connect many Ethernet hosts. Network hubs forward all received frames to all other network devices, and each host is responsible for filtering traffic. This is old-school Ethernet, which can be useful for debugging network issues.

Switches have largely supplanted hubs. Every Ethernet connection needs a unique identifier, called a *MAC address* (or sometimes an *Ethernet address*), which is a 48-bit number. Switches control the traffic sent to each host by filtering on the MAC and IP address of attached devices and (mostly) forwarding frames only to the devices they are meant for. Switching reduces the amount of traffic and load on each individual system by decreasing the amount of traffic each host must sort through.

On i386 and amd64 hardware, the MAC address is a property of the card. On some other platforms, such as SPARC, the MAC address is a property of the server itself. Both IPv4 and IPv6 use the MAC address to find other hosts on the local network.

IPv4 and ARP

When a system needs to transmit data to another IP-based host on the local Ethernet, it first broadcasts an Ethernet request asking, "Which MAC address is responsible for this IP address?" If a host responds, further data for that IP is transmitted to that MAC address. This process is handled by ARP.

Use `arp(8)` to view your system's ARP table, which is the list of hosts that your system knows. Enter `arp -a` to show all of the MAC addresses and IPv4 hostnames your computer knows.

```
$ arp -a
fly.blackhelicopters.org (192.0.2.225) at 00:a0:c8:10:eb:82 on fxp0
caddis.blackhelicopters.org (192.0.2.226) at 00:16:36:c0:58:a5 on fxp0 static
treble.blackhelicopters.org (192.0.2.227) at 00:0c:42:5a:58:ae on fxp0
salmon.blackhelicopters.org (192.0.2.232) at (incomplete) on fxp0
```

Here, you see the three hosts on my Ethernet network that this host has communicated with. I have more hosts, but because this machine hasn't spoken with them lately, they aren't in the local ARP table.

If a MAC address shows up as `incomplete`, your machine has attempted to communicate with this host but cannot get its MAC address. In this example, I've tried to send data to the host `salmon`, but my computer can't reach it. (Turning `salmon` back on would help.)

IPv6 and Neighbor Discovery

IPv6 hosts also use MAC addresses to find each other through ND, an IPv6 protocol introduced in the previous chapter. Interrogate your ND cache with `ndp(8)`. The command-line flags used for `ndp` are intentionally similar to those for `arp`.

```
$ ndp -a
Neighbor                                Linklayer Address  Netif Expire    S Flags
2001:db8:0:12:20c:29ff:feb5:7565        0:c:29:b5:75:65    vic0  permanent R
2001:db8:0:12:5446:fbc:fca0:f2e9        0:c:29:b5:75:65    vic0  permanent R
...
fe80::20c:29ff:feb5:7565%vic0           0:c:29:b5:75:65    vic0  permanent R
fe80::20c:42ff:fe20:7f42%vic0           0:c:42:20:7f:42    vic0  11h20m47s S R
fe80::1%lo0                             (incomplete)       lo0   permanent R
```

Like the ARP cache, the ND cache shows an IPv6 address, a physical address, the interface, and other details for each host. You'll see more ND entries than ARP entries because all of the link local addresses show up in the ND cache.

If you try to reach a host that is directly attached to your local network and it doesn't respond, check the ND cache. If an ND cache entry shows up as `(incomplete)`, as with ARP, there's some sort of basic connectivity issue.

Speed and Duplex

Ethernet supports a variety of speeds. The slowest speed you're likely to find today is 10 megabits per second (Mbps), but it's quickly disappearing. Most people use either 10/100Mbps or 1 gigabit per second (Gbps), although you'll see 10Gbps, 40Gbps, and 100Gbps Ethernet emerging.

The hosts and switch it's connected to on your network must agree on the speed of their connection. If the OpenBSD host thinks that it's connected at 100Mbps, but the switch thinks that the connection is 1Gbps, the connection will be flaky. While *autonegotiation* usually makes both sides agree on common settings (and is absolutely required for gigabit connections), you can manually set duplex and speed for 10/100Mbps connections. Although some switch vendors are notorious for poor autonegotiation, you should let your Ethernet configure itself whenever possible.

Duplex determines if a card can both transmit and receive data simultaneously. A *half-duplex* connection means that the Ethernet card is either transmitting or receiving at a given instant; it cannot do both. A *full-duplex* connection can both send and receive simultaneously. As with connection speed, if the switch and host disagree on the duplex setting, the connection will be flaky. Gigabit Ethernet connections involve much more than speed and duplex, and they *must* be autonegotiated.

Just because a device says that it can use the protocol defined as 10/100Mbps Ethernet doesn't mean that it can use that protocol with any speed. Also, a card labeled "1Gbps" might not actually pass a gigabit per second. Some network cards will pass their stated amount of traffic, while others will stagger and stumble at a few percent of that. Switch quality varies widely, too.

This may make more sense if you think of an Ethernet's stated speed as a language. For example, I could claim that I speak Russian and German, but I stopped studying foreign languages in 1985. When I went to Germany in 2007, I managed about three words a minute—with the aid of a translation card and phrase book. If I were an Ethernet card, the manufacturer would claim I spoke German and Russian, and ship me to Siberia.[1]

Get decent hardware. Don't ask on the OpenBSD mailing list, though. Someone has asked about hardware recommendations in the past few months. Check the archives. The advice hasn't changed.

Configuring Ethernet

When configuring Ethernet for client computers, if your IPv4 network offers DHCP, you should be able to plug right in. If you're using IPv6, you should be able to attach the cable and let autoconfiguration take over.

1. Many people offer to ship me to Siberia. But they all forget to include a return ticket. Strange.

If a particular machine will be a server, a static IP address probably makes more sense. Before assigning a static address, you'll need the following:

- An IP address (IPv4, IPv6 or both)
- The netmask/prefix length(s)
- The IP address(es) of the default gateway

Armed with this information, attach your system to the network and keep reading. I'll first discuss using `ifconfig(8)` and `route(8)` to perform changes manually, and then review how to set these automatically at boot. In any case, you must configure the resolver as discussed at the beginning of this chapter.

Using ifconfig(8)

If you installed OpenBSD over a network, your Ethernet connection should already be working, but it might not be set up exactly the way you like. To manage your network interfaces, use the `ifconfig(8)` tool.

Let's look at your Ethernet card and see what it has to say. Start by asking your system about all of the interfaces it has installed, by running `ifconfig`.

All OpenBSD systems have three logical interfaces out of the box: `lo0`, `enc0`, and `pflog0`. The `lo0` interface is the loopback interface, referring to the local machine. The `enc0` interface is an encapsulation interface, intended for IPsec traffic. Finally, `pflog0` is for logging PF traffic, as discussed in Chapter 22. The rest of the interfaces are physical ones.

Unlike some operating systems, OpenBSD network interfaces are named after the device driver of the underlying hardware. Here's a sample list:

```
$ ifconfig
fxp0: flags=8843<❶UP,BROADCAST,RUNNING,SIMPLEX,MULTICAST> mtu 1500
        lladdr 00:16:36:c0:58:a5
        priority: 0
        groups: egress
        media: Ethernet autoselect (100baseTX full-duplex)
     ❷ status: active
     ❸ inet 192.0.2.226 netmask 0xffffff00 broadcast 192.0.2.255
        inet6 2001:db8::216:36ff:fec0:58a5 prefixlen 64
        inet6 fe80::216:36ff:fec0:58a5%fxp0 prefixlen 64 scopeid 0x2
```

The interface `fxp0` uses the `fxp(4)` device driver, which the man page says is an Intel EtherExpress PRO 10/100 card. As you can see at ❶, the interface is up, meaning that it's active and ready to use. The `lladdr` is the link local address, or the MAC address of the card. This card is in the `egress` group. OpenBSD uses interface groups in several places, including the packet filter, as discussed in Chapter 22.

To see the type of physical media underlying the connection, check the `media` line. This particular connection runs at 100Mbps full-duplex. The connection is active, as shown at ❷; the physical layer has not only been

configured, but it also has a link light and is ready to go. The connection has been assigned an IPv4 address and netmask, as shown at ❸. You can see on the two lines that follow that both an IPv6 address and a link local IPv6 address have been assigned.

Use `ifconfig` to assign, change, or remove IP addresses from a network interface. The OpenBSD installer offers to configure your network cards at boot, but if you didn't configure all of your interfaces during installation, or if you add or remove network interfaces after installation, you will need to do so manually.

Adding an IP Address

To add an IP address for IPv4, start with the interface's assigned IP address and netmask.

```
# ifconfig interface-name IP-address netmask
```

For example, if your network card is `fxp0`, your IP address is 192.0.2.55, and the netmask is 255.255.255.128, you would run this:

```
# ifconfig fxp0 192.0.2.55 255.255.255.128
```

Specify the netmask in dotted-quad notation, hexadecimal, or even slash notation, like this:

```
# ifconfig fxp0 192.0.2.55/25
```

You don't need to specify a netmask separately if you use a slash.

Adding an IP address with IPv6 is a little different. Specify the address, a slash, and the prefix length, but don't try to add a separate netmask; just use the slash that's part of the address. Here's an example:

```
# ifconfig fxp0 inet6 2001:db8:0:12::2/64
```

Removing IP Addresses

If you need to remove an IP address from an interface, use the `delete` option of `ifconfig` for both IPv4 and IPv6 addresses.

```
# ifconfig fxp0 192.0.2.55 delete
```

The effect is immediate, so be sure you don't lock yourself out of the system by removing all of its reachable IP addresses, or by removing the only address your SSH daemon is attached to. (In certain rare cases, existing connections to deleted addresses might continue to work, but they probably won't, so don't count on it.)

Multiple IP Addresses on One Ethernet Card

One network interface can respond to requests for multiple IP addresses, which is important because a server might support hundreds or thousands of domains and need an IP address for each. (This isn't so important for plain websites, but it can be important for SSL-based websites and protocols that rely on reverse DNS.)

To add extra IP addresses to an interface, use *IP aliases*. IP aliases tell a network card to "answer requests for this IP address as well as your own." To add aliased IP addresses, use `ifconfig` with the keyword `alias` after the interface name to tell `ifconfig` this is an alias. Be sure to always use a netmask of 255.255.255.255, or /32, for alias addresses.

```
# ifconfig fxp0 alias 192.0.2.230/32
# ifconfig fxp0
...
        inet 192.0.2.226 netmask 0xfffffff0 broadcast 192.0.2.239
        inet 192.0.2.230 netmask 0xffffffff
```

The interface listed here has a main IP address of 192.0.2.226 and an alias IP address of 192.0.2.230.

When working with IPv6, add the `inet6` keyword, like this:

```
# ifconfig fxp0 inet6 alias 2001:db8:0:12::3/64
```

It's important to realize that all outgoing connections on a host with one network connection use the host's primary IP address. For example, you might have 2000 IP addresses bound to one interface, but when you `ssh` out, the connection comes from the primary address. Remember this when writing firewall rules and access control lists, because while some programs have an option to set a different source IP address, they're the exception.

The OpenBSD kernel doesn't really differentiate between the primary IP addresses and aliases—it just keeps a list of IP addresses—but it will use the first address on its list as the source address unless told otherwise. If a host has multiple network connections, the source address of outgoing connections is the main IP address of the network interface on which packets leave the system.

To remove an alias, use the `delete` option of `ifconfig` and give the IP address, without the netmask.

```
# ifconfig fxp0 delete 192.0.2.230
```

For IPv6, use `inet6 delete` instead.

```
# ifconfig fxp0 inet6 delete 2001:db8:0:12::3
```

NOTE *If you delete the main IP address on an interface, the first alias becomes the main IP address. If you have no IP address aliases remaining and you remove the interface's main IP address, that interface stops passing IP traffic.*